Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Çarpışma Algılama ve Fizik Temelleri

PyGame ile Oyun Geliştirme – Hafta 6

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

1. Rect Sınıfı ve Çarpışma
2. Sprite Çarpışma Fonksiyonları
3. Temel Fizik Simülasyonu
4. Çarpışma Tepkileri
5. Mini Proje: Breakout (Tuğla Kırma)
6. Uzay Kaçışı v2
7. Özet ve Kapanış

Rect Sınıfı ve Çarpışma

Rect Nedir?

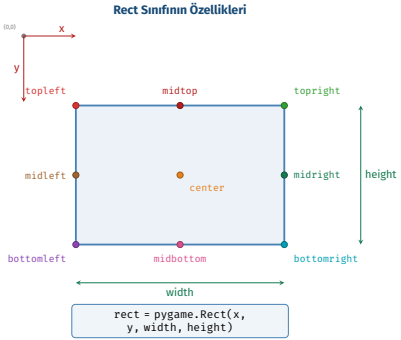
Tanım

pygame.Rect = Dikdörtgensel alan tanımlayan sınıf

- Konum (x, y) ve boyut (width, height)
- Her Sprite'ın `self.rect` özelliği bir Rect'tir
- Çarpışma algılamanın temel yapı taşı

Oluşturma Yolları

- `pygame.Rect(x, y, w, h)`
- `surface.get_rect()`
- `get_rect(center=(cx, cy))`



Rect'in konum ve boyut özellikleri

Rect Özellikleri

```
r = pygame.Rect(100, 50, 80, 60)

# Konum ozellikleri
r.x, r.y            # (100, 50)
r.top, r.bottom     # (50, 110)
r.left, r.right     # (100, 180)
r.center             # (140, 80)
r.centerx, r.centery
r.topleft, r.topright
r.bottomleft, r.bottomright
r.midtop, r.midbottom
r.midleft, r.midright

# Boyut ozellikleri
r.width, r.height   # (80, 60)
r.size              # (80, 60)
r.w, r.h            # kisa yol
```

Konum Özellikleri

- 16 farklı konum noktası
- Hepsi okunabilir **ve** yazılabilir
- `center` atayınca x, y otomatik güncellenir

Önemli

Rect özellikleri **tam sayı** tutar. Float atarsan yuvarlanır! Hassas konum için ayrı float değişken kullan.

Rect Yardımcı Metotları

```
# Tasima (kopya dondurur)
yeni = r.move(10, 5)

# Yerinde tasima
r.move_ip(10, 5)

# Boyut degistirme
buyuk = r.inflate(20, 20)
r.inflate_ip(20, 20)

# Sinir ici tutma
r.clamp_ip(ekran_rect)

# Ici bos mu?
r.normalize()
```

`_ip` Son Eki

- `move()` → yeni Rect döndürür
- `move_ip()` → mevcut Rect'i değiştirir
- "in place" = yerinde değiştir
- `inflate`, `clamp` için de geçerli

`inflate()` Kullanımı

`inflate(20, 20)` → her yönden 10px büyüt.
`inflate(-10, -10)` → daralt.
Çarpışma kutusunu ayarlamak için ideal!

colliderect() – Dikdörtgen Çarpışması

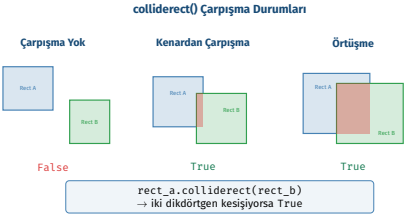
Nasıl Çalışır?

`rect1.colliderect(rect2)` → `True` / `False`

- İki dikdörtgen örtüşüyor mu?
- AABB (Axis-Aligned Bounding Box) testi
- En hızlı çarpışma yöntemi
- Kenar temas da `True` döndürür

Sınırlamalar

- Sadece dikdörtgen alanı kontrol eder
- Daire veya düzensiz şekiller için hassas değil
- Döndürülmüş nesnelerde kutu büyür



Çarpışma durumları: örtüşen ve örtüşmeyen

colliderect() – Kod Örneği

```
# Iki rect tanimla
oyuncu = pygame.Rect(100, 100, 50, 50)
dusman = pygame.Rect(130, 120, 40, 40)

# Carpisma kontrolu
if oyuncu.colliderect(dusman):
    print("Carpisma!")
    can -= 1

# Liste ile kontrol
for dusman in dusman_listesi:
    if oyuncu.colliderect(dusman):
        dusman.kill()
        skor += 10
```

Performans

- C ile yazılmış → çok hızlı
- 1000 nesne bile sorun değil
- Piksel testi gerektirmez

Kullanım Alanları

- Oyuncu–düşman teması
- Mermi–hedef kontrolü
- Güç artırıcı toplama
- Zemin çarpışması

collidepoint() – Nokta Çarpışması

```
# Fare tiklamasi ile Rect kontrolu
for olay in pygame.event.get():
    if olay.type == pygame.MOUSEBUTTONDOWN
        :
        # Fare noktasi rect icinde mi?
        if buton.collidepoint(olay.pos):
            print("Butona tiklandi!")

# Pratik kullanim: Menu butonlari
menu_butonlari = [
    pygame.Rect(300, 200, 200, 50),
    pygame.Rect(300, 270, 200, 50),
    pygame.Rect(300, 340, 200, 50),
]
for i, buton in enumerate(menu_butonlari):
    if buton.collidepoint(fare_pos):
        # Uzerine gelince renk degistir
        renk = VURGU_RENK
```

`collidepoint(x, y)`

- Bir nokta Rect içinde mi?
- Fare tıklaması kontrolü
- Menü butonları için ideal
- Tuple da kabul eder: `collidepoint(pos)`

Diğer Collide Metotları

- `collidelist()` → liste ile kontrol
- `collidelistall()` → tüm çarpışmalar
- `collidedict()` → sözlük ile

Sprite Çarpışma Fonksiyonları

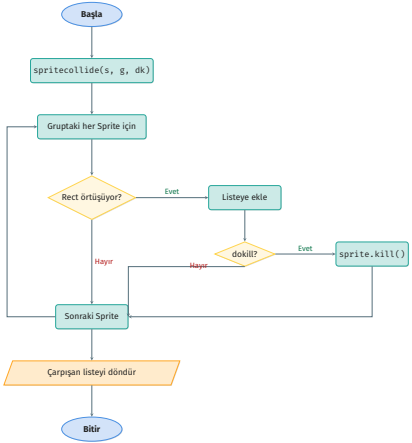
Sprite Çarpışma Sistemi

Neden Sprite Çarpışma Fonksiyonları?

- `colliderect()` tek tek kontrol gerektirir
- Sprite gruplarıyla toplu kontrol daha verimli
- PyGame yerleşik fonksiyonları optimizedir
- `kill()` ile otomatik temizleme desteği

3 Temel Fonksiyon

1. `spritecollide()` – Sprite vs Grup
2. `groupcollide()` – Grup vs Grup
3. `spritecollideany()` – Hızlı tekli kontrol



Sprite çarpışma kontrol akışı

spritecollide() – Sprite vs Grup

```
# Temel kullanim
carpisan_dusmanlar = pygame.sprite.
    spritecollide(
    oyuncu,        # Tek sprite
    dusmanlar,     # Hedef grup
    True           # dokill: True = sil
)

# Sonuc: carpisan sprite listesi
for dusman in carpisan_dusmanlar:
    skor += dusman.puan
    patlama_efekti(dusman.rect.center)

# dokill=False: sadece tespit et
yakin_dusmanlar = pygame.sprite.
    spritecollide(
    oyuncu, dusmanlar, False
)
if yakin_dusmanlar:
    uyari_goster()
```

Parametreler

- **sprite** – kontrol edilecek sprite
- **group** – hedef grup
- **dokill** – `True` ise çarpışan sprite'ları otomatik sil

`dokill` Parametresi

`True` → çarpışan sprite `kill()` edilir
`False` → sadece liste döndürülür

Mermi çarpışması → `True`
Hasar kontrolü → `False`

groupcollide() – Grup vs Grup

```
# Mermiler vs Dusmanlar
carpismalar = pygame.sprite.groupcollide(
    mermiler,      # Grup 1
    dusmanlar,     # Grup 2
    True,          # dokill1: mermiyi sil
    True           # dokill2: dusmani sil
)

# Sonuc: dict {mermi: [dusman_listesi]}
for mermi, vurulan_liste in carpismalar.
    items():
    for dusman in vurulan_liste:
        skor += dusman.puan
        Patlama(dusman.rect.center)
```

Döndürülen Sözlük

- Anahtar: Grup 1'den çarpışan sprite
- Değer: Grup 2'den çarpışan sprite listesi
- Boş sözlük = çarpışma yok

Kullanım Senaryoları

- Mermi ↔ düşman
- Düşman mermi ↔ oyuncu
- Nesne ↔ duvar
- Top ↔ tuğla (Breakout)

Dikkat

Her iki grup için ayrı `dokill` parametresi! Mermi silinsin ama düşman kalabilir (hasar sistemi).

Çarpışma Yöntemleri – collided Parametresi

```
# Varsayilan: Rect (AABB) carpisma
pygame.sprite.spritecollide(
    oyuncu, dusmanlar, False
)  # collided=None -> collide_rect

# Daire carpisma
pygame.sprite.spritecollide(
    oyuncu, dusmanlar, False,
    pygame.sprite.collide_circle
)

# Piksel tabanlı (mask) carpisma
pygame.sprite.spritecollide(
    oyuncu, dusmanlar, True,
    pygame.sprite.collide_mask
)
```

3 Çarpışma Yöntemi

Yöntem	Hız
`collide_rect`	En hızlı
`collide_circle`	Orta
`collide_mask`	En yavaş

collide_mask İçin

- Sprite'a `self.mask` gerekli
- `mask.from_surface(image)`
- Alpha kanalından otomatik maske oluşturur
- Piksel düzeyinde hassasiyet

Mask Çarpışma – Piksel Hassasiyeti

```python
class Gemi(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.image.load(
            "gemi.png"
        ).convert_alpha()
        self.rect = self.image.get_rect(
            center=(x, y)
        )
        # Piksel maskesi olustur
        self.mask = pygame.mask.from_surface
            (
            self.image
        )
```

Ne Zaman Kullanmalı?

- Düzensiz şekilli sprite'lar
- Boss savaşlarında hassas vuruş

Rect vs Mask Karşılaştırma

Rect (AABB):

- Hızlı ama kaba
- Boş alanlarda yanlış pozitif

Mask (Piksel):

- Yavaş ama hassas
- Sadece opak pikseller sayılır

Performans İpucu

Önce `collide_rect` ile ön eleme, sonra yakın olanlara `collide_mask` uygula.

Temel Fizik Simülasyonu

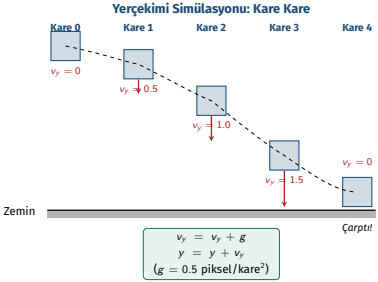
Yerçekimi Simülasyonu

Gerçek Dünya Fiziği

- Yerçekimi = sabit ivme (9.8 m/s^2)
- Her karede hıza ivme eklenir
- Hız konuma eklenir
- Sonuç: doğal düşme hareketi

Oyun Fiziği Formülleri

- $v_y = v_y + g \times dt$
- $y = y + v_y \times dt$
- g = yerçekimi sabiti (piksel/kare^2)
- Oyunlarda genelde $g = 0.5 \sim 1.0$



Yerçekimi altında düşüş adımları

Yerçekimi Uygulaması

```python
YERCEKIMI = 0.5
ZEMIN_Y = 550

class Nesne(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((30, 30))
        self.image.fill((200, 80, 80))
        self.rect = self.image.get_rect(
            center=(x, y)
        )
        self.hiz_y = 0  # Dikey hiz

    def update(self):
        # Yercekimi uygula
        self.hiz_y += YERCEKIMI
        self.rect.y += self.hiz_y

        # Zemin kontrolu
        if self.rect.bottom >= ZEMIN_Y:
            self.rect.bottom = ZEMIN_Y
            self.hiz_y = 0
```

Adım Adım

1. Hıza yerçekimi ekle
2. Konuma hız ekle
3. Zemin kontrolü yap
4. Zemindeyse hızı sıfırla

Zıplama Eklemek

Yukarı hız ver:

- `self.hiz_y = -12`
- Yerçekimi otomatik geri çeker
- Doğal parabol oluşur

Hız ve Momentum

```
class Top(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((20, 20))
        pygame.draw.circle(
            self.image, (255, 200, 0),
            (10, 10), 10
        )
        self.image.set_colorkey((0, 0, 0))
        self.rect = self.image.get_rect(
            center=(x, y)
        )
        # 2D hiz vektoru
        self.hiz_x = 4
        self.hiz_y = -3

    def update(self):
        self.hiz_y += YERCEKIMI
        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y
```

2D Hız Vektörü

- `hiz_x` – Yatay hareket
- `hiz_y` – Dikey hareket
- Birlikte → eğik hareket
- Momentum = kütle × hız

Fırlatma Fiziği

- Başlangıç hızı ver (x, y)
- Yerçekimi sadece y'yi etkiler
- x sabit kalır (sürtünme yoksa)
- Sonuç: parabol yörünge

Sürtünme

```python
SURTUNME = 0.98  # 0-1 arasi carpan
HAVA_DIRENCI = 0.995

class FizikNesne(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        # ... init kodu ...
        self.hiz_x = 10
        self.hiz_y = 0
        self.zeminde = False

    def update(self):
        self.hiz_y += YERCEKIMI

        # Surtunme uygula
        if self.zeminde:
            self.hiz_x *= SURTUNME
        else:
            self.hiz_x *= HAVA_DIRENCI

        # Cok kucuk hizi sifirla
        if abs(self.hiz_x) < 0.1:
            self.hiz_x = 0

        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y
```

Sürtünme Modeli

- Hızı her karede çarpanla azalt
- 0.98 = hafif sürtünme (buz)
- 0.85 = güçlü sürtünme (çim)
- 1.0 = sürtünme yok

Farklı Zeminler

Buz	0.99
Normal	0.95
Çim	0.85
Kum	0.75

Fizik Motoru Yapısı

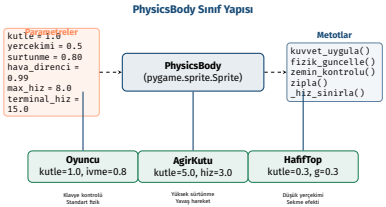
Basit Fizik Motoru

Tüm fizik hesaplamalarını tek sınıfta topla:

1. Kuvvetleri topla (yerçekimi, itme, rüzgar)
2. İvmeyi hesapla ($a = F/m$)
3. Hızı güncelle ($v = v + a$)
4. Sürtünme uygula ($v = v \times f$)
5. Konumu güncelle ($p = p + v$)
6. Çarpışma kontrolü

Neden Fizik Motoru?

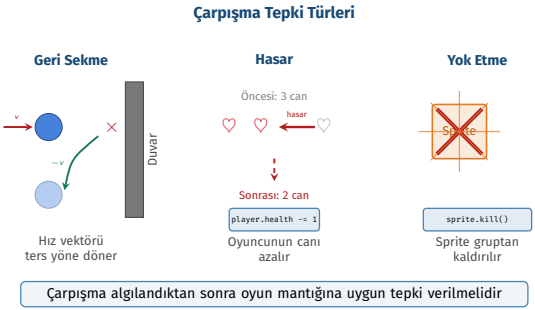
Her sprite'a ayrı ayrı fizik yazmak yerine, merkezi bir sistem ile tutarlı davranış sağlanır.



Fizik motoru işlem döngüsü

Çarpışma Tepkileri

Çarpışma Tepki Türleri



Farklı çarpışma tepki senaryoları

Tepki Türleri

1. **Geri Sekme** – Top, mermi
2. **Hasar** – Can azaltma
3. **Yok Etme** – kill()
4. **Toplama** – Güç artırıcı
5. **İtme** – Fiziksel tepki

Tasarım Kararı

Aynı çarpışma farklı tepkiler doğurabilir. Oyun tasarımına göre seçim yap!

Geri Sekme – Duvar ve Kenar

```
class SekenTop(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((20, 20))
        pygame.draw.circle(
            self.image, (255, 255, 0),
            (10, 10), 10
        )
        self.image.set_colorkey((0, 0, 0))
        self.rect = self.image.get_rect(
            center=(400, 300)
        )
        self.hiz_x = 5
        self.hiz_y = 4

    def update(self):
        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y
        # Yatay sekme
        if self.rect.left < 0 or \
           self.rect.right > 800:
            self.hiz_x *= -1
        # Dikey sekme
        if self.rect.top < 0 or \
           self.rect.bottom > 600:
            self.hiz_y *= -1
```

Sekme Mekaniği

- Hız yönünü ters çevir
- Yatay çarpışma → `hiz_x *= -1`
- Dikey çarpışma → `hiz_y *= -1`
- Enerji kaybı: `hiz *= 0.9`

Enerji Kaybı

Gerçekçi sekme için her sekmede hız azalt:

- `hiz_y *= -0.8`
- Her sekmede %20 enerji kaybı
- Sonunda top durur

Hasar Sistemi

```python
class Oyuncu(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        # ... init kodu ...
        self.max_can = 100
        self.can = self.max_can
        self.hasar_bekleme = 0  # i-frame

    def hasar_al(self, miktar):
        if self.hasar_bekleme <= 0:
            self.can -= miktar
            self.hasar_bekleme = 60  # 1 sn
            if self.can <= 0:
                self.can = 0
                self.kill()

    def update(self):
        if self.hasar_bekleme > 0:
            self.hasar_bekleme -= 1
            # Yanıp sönme efekti
            if self.hasar_bekleme % 10 < 5:
                self.image.set_alpha(100)
            else:
                self.image.set_alpha(255)
```

Hasar Sistemi Bileşenleri

- `can` – Mevcut can puanı
- `max_can` – Maksimum can
- `hasar_al()` – Hasar metodu
- `hasar_bekleme` – Dokunulmazlık

i-frame (Invincibility)

Hasar aldıktan sonra kısa süre dokunulmaz. Oyuncunun art arda hasar almasını önler. Yanıp sönme efekti ile görsel geri bildirim sağla.

Yok Etme ve Patlama Efekti

```python
# Ana dongude carpisma isleme
carpismalar = pygame.sprite.groupcollide(
    mermiler, dusmanlar, True, False
)
for mermi, vurulanlar in carpismalar.items():
    for dusman in vurulanlar:
        dusman.can -= mermi.hasar
        if dusman.can <= 0:
            # Patlama efekti olustur
            patlama = Patlama(
                dusman.rect.center
            )
            efektler.add(patlama)
            tum_spritelar.add(patlama)
            # Dusmani yok et
            dusman.kill()
            skor += dusman.puan
```

Yok Etme Akışı

1. Çarpışma tespit et
2. Hasar uygula
3. Can 0 ise → patlama oluştur
4. `kill()` ile sprite'ı sil
5. Skor güncelle

Güç Artırıcı Toplama

Aynı mantık, farklı tepki:

- Çarpışma → güç artır
- `kill()` ile topla
- Ses efekti çal

Mini Proje: Breakout (Tuğla Kırma)

Breakout – Proje Tanıtımı

Oyun Konsepti

- Raket ile topu kontrol et
- Top tuğlalara çarparak onları kır
- Tüm tuğlaları kırınca seviye tamamlanır
- Top alt kenardan çıkarsa can kaybedersin

Kullanılan Kavramlar

- Rect çarpışması (`colliderect`)
- Geri sekme fiziği
- Sprite grupları ve `kill()`
- Hız vektörleri

Oyun Nesneleri

- **Raket** – Fare/klavye ile hareket
- **Top** – Hız vektörü ile sekme
- **Tuğla** – Grid düzeni, farklı renkler
- **Duvar** – Kenar sınırları

Breakout – Tuğla ve Raket

```python
class Tugla(pygame.sprite.Sprite):
    def __init__(self, x, y, renk):
        super().__init__()
        self.image = pygame.Surface((75, 25))
        self.image.fill(renk)
        self.rect = self.image.get_rect(
            topleft=(x, y))

# Tugla grid olustur
renkler = [(255,0,0),(255,127,0),
           (255,255,0),(0,255,0),(0,0,255)]
for satir in range(5):
    for sutun in range(10):
        tugla = Tugla(sutun*80+2,
            satir*30+50, renkler[satir])
        tuglalar.add(tugla)

class Raket(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((100, 15))
        self.image.fill((200, 200, 200))
        self.rect = self.image.get_rect(
            center=(400, 560))
    def update(self):
        self.rect.centerx = pygame.mouse.get_pos()[0]
        self.rect.clamp_ip(ekran_rect)
```

Tuğla Grid Sistemi

- 5 satır $\times$ 10 sütun $=$ 50 tuğla
- Her satır farklı renk
- Sprite Group ile yönetim
- `kill()` ile kırılma

Raket Kontrolü

- Fare x konumunu takip et
- `clamp_ip()` ile sınırda tut
- Sadece yatay hareket

Breakout – Top Fiziği

```
class Top(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((16, 16))
        pygame.draw.circle(
            self.image, (255, 255, 255),
            (8, 8), 8
        )
        self.image.set_colorkey((0, 0, 0))
        self.rect = self.image.get_rect(
            center=(400, 540)
        )
        self.hiz_x = 4
        self.hiz_y = -4

    def update(self):
        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y
        # Duvar sekmesi
        if self.rect.left < 0 or \
           self.rect.right > 800:
            self.hiz_x *= -1
        if self.rect.top < 0:
            self.hiz_y *= -1
        # Alt kenar = can kaybi
        if self.rect.top > 600:
            self.kill()
```

Top Mekaniği

- 2D hız vektörü ile hareket
- Yan + üst duvarlarda sekme
- Alt kenar → can kaybı
- Raket ile sekme (açı ayarı)

Açılı Sekme

Topun rakete çarpma noktasına göre açısını değiştir:

- Raket ortası → düz
- Raket kenarı → açılı
- Oyunu daha stratejik yapar

Uzay Kaçışı v2

Uzay Kaçışı v2 – Yeni Özellikler

Aşamalı Geliştirme

Versiyon	Bölüm	Özellikler
v1	Bölüm 5	Oyuncu + yıldız arka planı
v2	Bölüm 6	Düşmanlar + mermiler + çarpışma
v3	Bölüm 7	Ses efektleri + müzik
v4	Bölüm 8	Animasyon + patlama + zamanlayıcı

v2 Eklemeleri

- Düşman sprite'ları
- Mermi ateşleme (Space tuşu)
- Mermi–düşman çarpışması
- Oyuncu–düşman çarpışması
- Skor ve can sistemi

Kullanılan Fonksiyonlar

- `spritecollide()` – oyuncu kontrolü
- `groupcollide()` – mermi kontrolü
- `kill()` – yok etme
- Hasar + dokunulmazlık sistemi

Uzay Kaçışı v2 – Çarpışma Kodu

```python
# Gruplar
tum_spritelar = pygame.sprite.Group()
dusmanlar = pygame.sprite.Group()
mermiler = pygame.sprite.Group()

# Mermi atesleme (ana dongude)
if olay.type == pygame.KEYDOWN:
    if olay.key == pygame.K_SPACE:
        mermi = Mermi(oyuncu.rect.centerx,
                      oyuncu.rect.top)
        mermiler.add(mermi)
        tum_spritelar.add(mermi)

# Carpisma kontrolleri
# 1. Mermi vs Dusman
carpismalar = pygame.sprite.groupcollide(
    mermiler, dusmanlar, True, True
)
for mermi, vurulanlar in carpismalar.items():
    skor += len(vurulanlar) * 10

# 2. Oyuncu vs Dusman
vuranlar = pygame.sprite.spritecollide(
    oyuncu, dusmanlar, True
)
for dusman in vuranlar:
    oyuncu.hasar_al(20)
```

Çarpışma Akışı

1. Space → mermi oluştur
2. `groupcollide()` → mermi–düşman
3. `spritecollide()` → oyuncu–düşman
4. Skor ve can güncelle

Performans

- Mermileri ekran dışında `kill()`
- Düşmanları alt kenardan çıkınca `kill()`
- Gereksiz sprite birikmesini önle

Özet ve Kapanış

Bu Hafta Öğrendiklerimiz

Rect Çarpışma

- `Rect` özellikleri
- `colliderect()`
- `collidepoint()`
- AABB testi

Sprite Çarpışma

- `spritecollide()`
- `groupcollide()`
- `collide_mask`
- `dokill` parametresi

Fizik Temelleri

- Yerçekimi
- Hız vektörleri
- Sürtünme
- Fizik motoru

Çarpışma Tepkileri

- Geri sekme
- Hasar sistemi
- Yok etme + efekt
- Breakout projesi

Gelecek Hafta: Ses ve Müzik – Oyunlarımıza ses efektleri ve arka plan müziği ekliyoruz!

Lab Çalışması: Uzay Kaçışı v2 + Breakout

Görev 1: Uzay Kaçışı v2

1. v1 projesine düşman sprite'ları ekle
2. Space tuşu ile mermi ateşleme
3. `groupcollide()` ile mermi–düşman çarpışması
4. `spritecollide()` ile oyuncu–düşman çarpışması
5. Skor ve can sistemi

Görev 2: Breakout

1. Raket (fare ile kontrol)
2. Seken top (hız vektörü)
3. 5×10 tuğla grid
4. Top–tuğla çarpışma ve yok etme
5. Skor ve 3 can sistemi

Bonus

- Farklı düşman tipleri (zigzag, takipçi)
- Güç artırıcılar (kalkan, çift mermi)
- Breakout'ta farklı tuğla dayanıklılıkları

Ödev

- Kitaptan Bölüm 6'yı okuyun
- Lab egzersizlerini tamamlayın
- Uzay Kaçışı v2 projesini bitirin:
  1. Mermi ateşleme sistemi
  2. Düşman–mermi çarpışması (`groupcollide`)
  3. Can sistemi ve dokunulmazlık (i-frame)
  4. Skor gösterimi
- Breakout projesinde en az 3 seviye tasarlayın

Kaynaklar

`pygame.org/docs/ref/rect.html` – Rect modülü
`pygame.org/docs/ref/sprite.html` – Sprite çarpışma fonksiyonları
`pygame.org/docs/ref/mask.html` – Mask modülü

Sorular?

Teşekkürler!